

CIC – 209: DATA STRUCTURES

Unit – I Lists

List ADT

- List is an
 - Ordered sequence of data items called elements
 - $A_0, A_1, A_2, A_3, \dots, A_{N-1}$ is a list of size N
 - size of an empty list is 0
 - A_{i+1} succeeds A_i
 - A_{i-1} preceeds A_i
 - Position of A_i is i
 - First element is A_0 called “head”
 - Last element is A_{N-1} called “tail”

Operations on Lists

- Make Empty (Create List)
- Print List (Traversing)
- Find (Searching)
- Find K^{th}
- Insert (Insertion)
- Delete (Deletion)
- Next
- Previous
- Sort (Sorting)

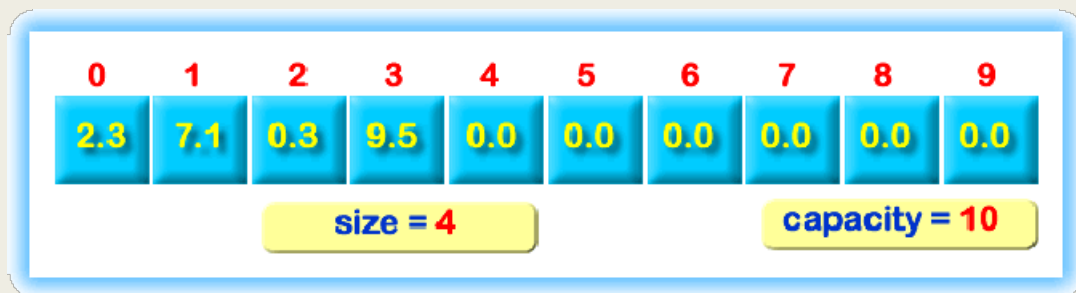
List – An Example

- The elements of a list are 34, 12, 52, 16, 12
 - *Find (52) -> 3*
 - *Insert (20, 4) -> 34, 12, 52, 20, 16, 12*
 - *Delete (52) -> 34, 12, 20, 16, 12*
 - *FindKth (3) -> 20*

List Implementation

- Lists can be implemented using:
 - *Arrays*
 - *Linked List*
- There are two possibilities:
 - *Ordered List*
 - *Unordered List*

List Implementation using Array



Insertion into an Ordered List



Operations Running Times

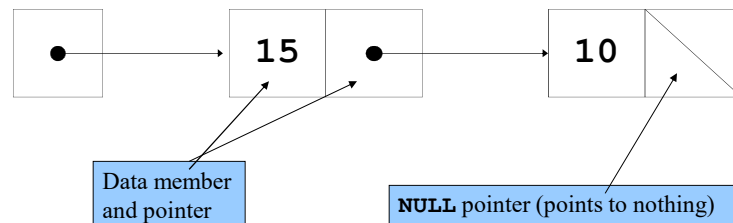
PrintList	$O(N)$
Find	
Insert	$O(N)$ (on average half
Delete	needs to be moved)
FindKth	
Next	$O(1)$
Previous	

Disadvantages of using Arrays

- Need to define a size for array
 - *High overestimate (waste of space)*
- insertion and deletion is very slow
 - *need to move elements of the list*
- redundant memory space
 - *it is difficult to estimate the size of array*

Self Referential Structures

- Self-referential structures
 - Structure that contains a pointer to a structure of the same type
 - Can be linked together to form useful data structures such as lists, queues, stacks and trees
 - Terminated with a **NULL** pointer (0)
- Diagram of two self-referential structure objects linked together



Self-Referential Structures (Contd...)

```
struct node
{
    int data;
    struct node *nextPtr;
}
```

- **nextPtr**
 - Points to an object of type node
 - Referred to as a link
 - Ties one **node** to another **node**

Dynamic Memory Allocation

- Dynamic memory allocation
 - Obtain and release memory during execution
- **malloc**
 - Takes number **sizeof** of bytes to allocate
 - Use to determine the size of an object
 - Returns pointer of type **void ***
 - A **void *** pointer may be assigned to any pointer
 - If no memory available, returns **NULL**
 - Example


```
newPtr = malloc( sizeof( struct node ) );
```
- **free**
 - Deallocates memory allocated by **malloc**
 - Takes a pointer as an argument
 - **free (newPtr);**

Linked Lists

- The Linked list is a chain of structures in which each structure consists of data as well as pointer, which stores the address (link) of the next logical structure in the list.
- A linked list is a data structure used to maintain a dynamic series of data.
- Think of a linked list as a line of bogies of train where each bogie is connected on to the next bogie. If you know where the first bogie is, you can follow its link to the next one. By following links, you can find any bogie of the train. When you get to a bogie that isn't holding (linked) on to another bogie, you know you are at the end.
- Linked lists work in the same way, except programmers usually refer to nodes instead of bogies.
- A single node is defined in the same way as any other user defined type or object, except that it also contains a pointer to a variable of the same type as itself.

Linked Lists

- **Linked list**
 - Linear collection of self-referential objects, called nodes
 - Connected by pointer links
 - Accessed via a pointer to the first node of the list
 - Subsequent nodes are accessed via the link-pointer member of the current node
 - Link pointer in the last node is set to null to mark the list's end
 - It is a dynamic data structure.
 - Linked lists allocate memory for each element separately and only when necessary
 - The data items in the linked list are not in consecutive memory locations.
- **Use a linked list instead of an array when**
 - You have an unpredictable number of data elements
 - Your list needs to be sorted quickly

Advantages of Linked Lists

- Linked lists are dynamic data structures. i.e., they can grow or shrink during the execution of a program.
- Linked lists have efficient memory utilization. Here, memory is not pre-allocated. Memory is allocated whenever it is required and it is de-allocated (removed) when it is no longer needed.
- Insertion and Deletions are easier and efficient. Linked lists provide flexibility in inserting a data item at a specified position and deletion of the data item from the given position.
- Many complex applications can be easily carried out with linked lists.

Disadvantages of Linked Lists

- It consumes more space because every node requires a additional pointer to store address of the next node.
- Searching a particular element in list is difficult and also time consuming.

Comparison between Array and Linked Lists

ARRAY	LINKED LIST
Size of an array is fixed (i.e. static)	Size of the list is not fixed (i.e. dynamic)
Memory is allocated from stack	Memory is allocated from heap
It is necessary to specify the number of elements during declaration (i.e., during compile time).	It is not necessary to specify the number of elements during declaration (i.e., memory is allocated during run time).
It occupies less memory than a linked list for the same number of elements.	It occupies more memory.
Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.	Inserting a new element at any position can be carried out easily.
Deleting an element from an array is not possible.	Deleting an element is possible.
Both sequential and random access are efficient.	Both sequential access is efficient by random access is not.
Rearranging of elements is inefficient.	Rearranging of elements is efficient.
There is no overhead per element.	There is overhead of 1 or 2 links per element.

Applications of Linked Lists

- Linked lists are used to represent and manipulate polynomial. Polynomials are expression containing terms with non zero coefficient and exponents. For example:

$$P(x) = a_0 X^n + a_1 X^{n-1} + \dots + a_{n-1} X + a_n$$
- Represent very large numbers and operations of the large number such as addition, multiplication and division.
- Linked lists are to implement stack, queue, trees and graphs.
- Implement the symbol table in compiler construction

Types Linked Lists

- **Singly linked list**
 - Begins with a pointer to the first node
 - Terminates with a null pointer
 - Only traversed in one direction
- **Circular, singly linked**
 - Pointer in the last node points back to the first node
- **Doubly linked list**
 - Two “start pointers” – first element and last element
 - Each node has a forward pointer and a backward pointer
 - Allows traversals both forwards and backwards
- **Circular, doubly linked list**
 - Forward pointer of the last node points to the first node and backward pointer of the first node points to the last node

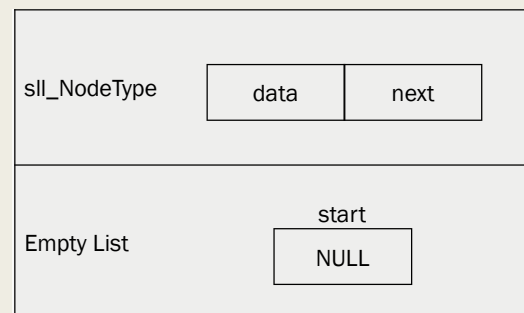
Singly Linked List (SLL)

- Each node contains two fields; (i) a "data" field to store whatever element, and (ii) a "next pointer" field which is a pointer used to link or connect to the next node.
- Each node is allocated in the heap using malloc(), so the node memory continues to exist until it is explicitly de-allocated using free(). The Head of the list is a pointer to the “start / first” node of the list.

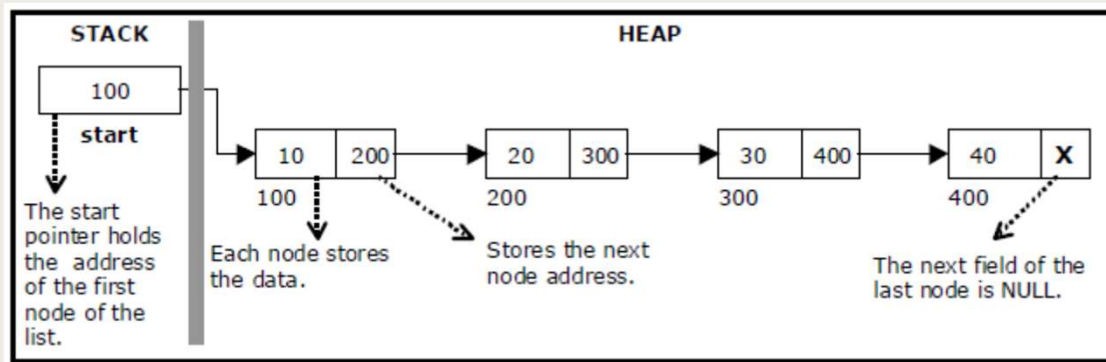
```

struct sll_Node
{
    int data;
    struct sll_Node *next;
};
typedef struct sll_Node sll_NodeType;
sll_NodeType *start = NULL;

```



Example of Singly Linked List



Singly Linked List – Basic Operations

- Creation
- Insertion
- Deletion
- Traversing
- Searching

Creating Node

Algorithm:

1. Allocate memory for the **structure of NodeType** and assign it to **newnode**.
2. Store the information in **data** field of the **newnode**.
3. Set the **next pointer** to **NULL**
4. Return the **address of the newnode**.
5. End

newnode	
1020	X
100	

Creation of Empty List

Algorithm:

1. Declare **start** pointer of the structure **NodeType**.
2. Set the **start** pointer with **NULL**.
3. End

Insertion of a Node

Steps:

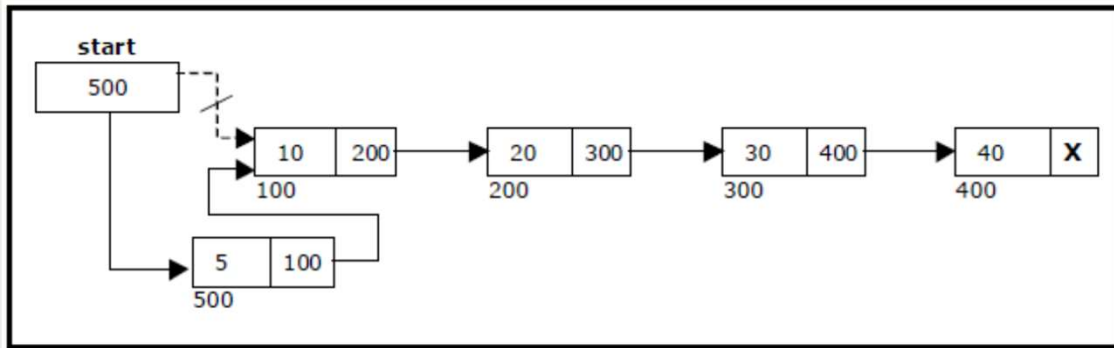
1. Create **newnode**.
2. If the list is empty (i.e. start is NULL) then assign the newnode to start.
3. else, if the newnode comes before the start node then
 - (a) *Set the next pointer of newnode with start*
 - (b) *Assign the newnode to start*
4. else, traverse the list to reach at the node after which the newnode to be inserted and perform the following operations:
 - (a) *Set the next pointer of newnode with the next pointer of the node*
 - (b) *Assign the newnode to the next pointer of the node*
5. End

Insertion of a Node

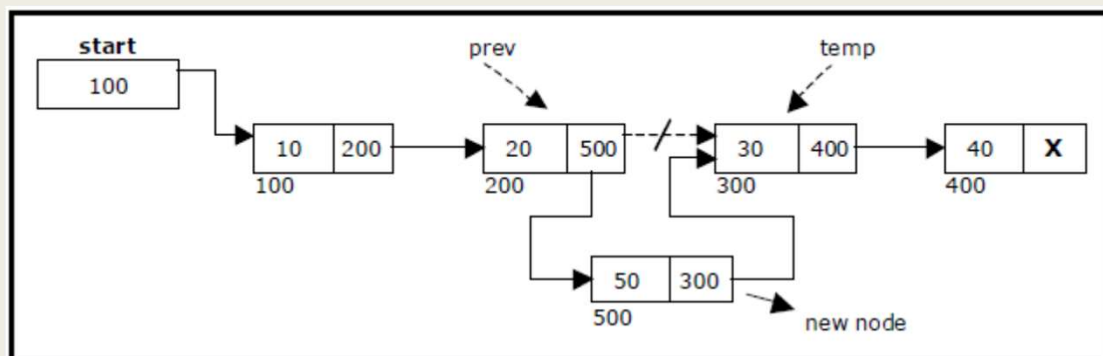
Algorithm:

1. newnode = CreateNode(data)
2. If start is NULL then start = newnode
3. else if the newnode comes before the start node then
 - (a) *newnode->next = start*
 - (b) *start = newnode*
4. else, traverse the list to reach at the node (i.e. curr_node) after which the newnode to be inserted and perform the following operations:
 - (a) *newnode->next = curr_node->next*
 - (b) *curr_node->next = newnode*
5. End

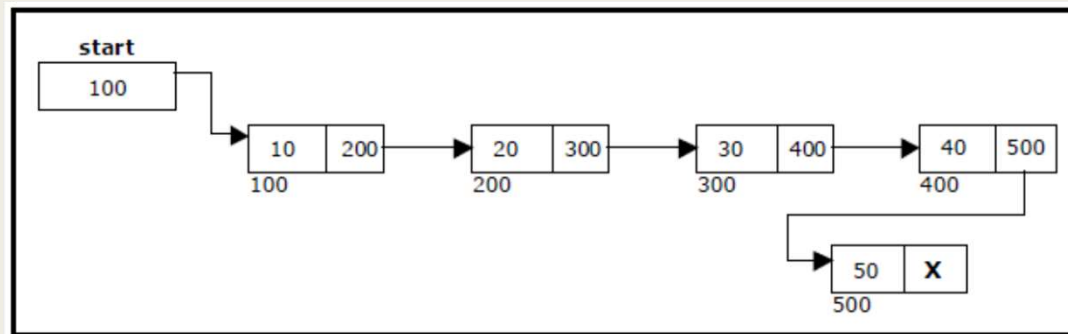
Insertion at the Beginning



Insertion at Intermediate Position



Insertion at the End



Deletion of a Node

Steps:

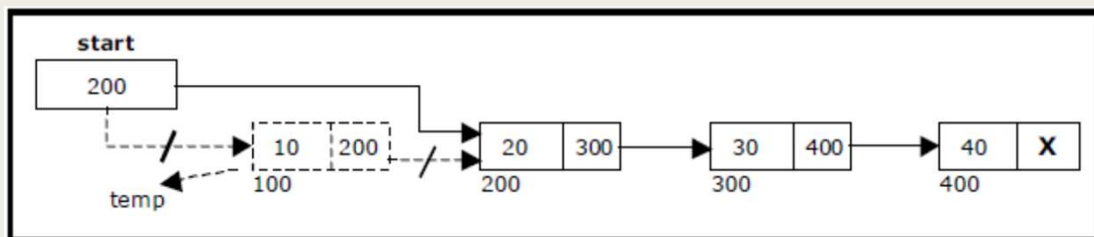
1. If start is NULL then Print "List is Empty" and Go to Step 4
2. Else If First node to be deleted then
 - (a) Assign the start to oldnode
 - (b) Assign next pointer of the oldnode to start
 - (c) return oldnode
3. Else traverse to the list and reach to the Node (i.e. curr_node) just before the node to be deleted and perform:
 - (a) Assign the next pointer of the curr_node to oldnode
 - (b) Assign next pointer of the oldnode to curr_node
 - (c) return oldnode
4. End.

Deletion of a Node

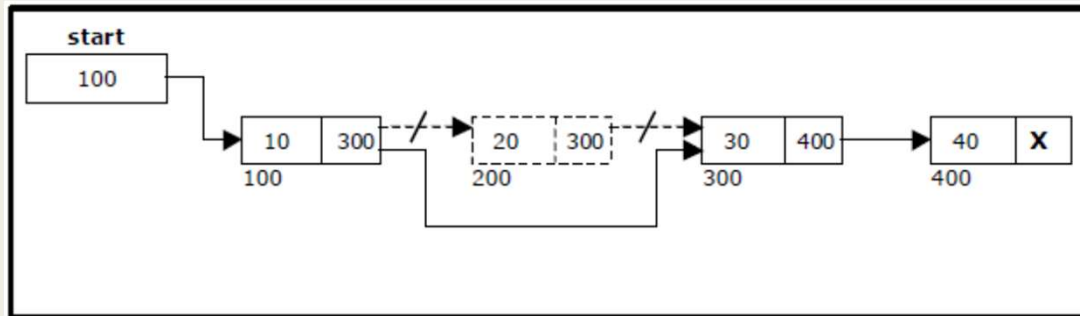
Algorithm:

1. If start is NULL then Print "List is Empty" and Go to Step 7
2. Else If First node to be deleted then
 - (a) *oldnode* = *start*
 - (b) *start* = *oldnode->next*
3. Else traverse to the list and reach to the Node (i.e. *curr_node*) just before the node to be deleted and perform:
 - (a) *oldnode* = *curr_node->next*
 - (b) *curr_node->next* = *oldnode->next*
4. *data* = *oldnode->data*
5. *free* (*oldnode*)
6. *return data*
7. End.

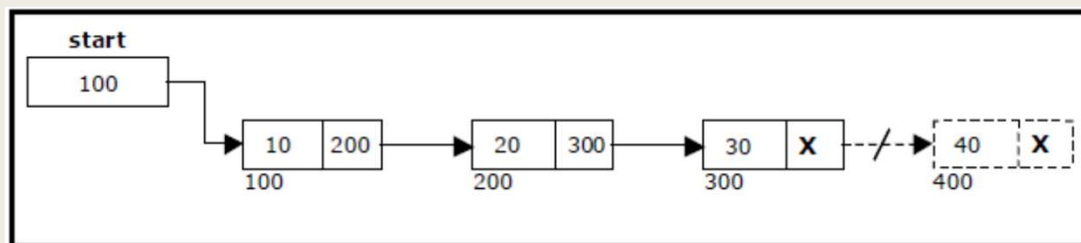
Deletion from the Beginning



Deletion from Intermediate Position



Deletion from the End



Traversing

Algorithm:

1. If start is NULL then Print "List is Empty" and Go to Step 4
2. Else
 - (a) Create a new node named temp of the structure NodeType
 - (b) Assign the start to temp
3. Repeat until temp is NULL
 - (a) Print data of the node temp
 - (b) Move to the next node by assigning next pointer of the temp to temp itself.
4. End.

Counting the Number of Nodes (i.e. Size of List)

Algorithm:

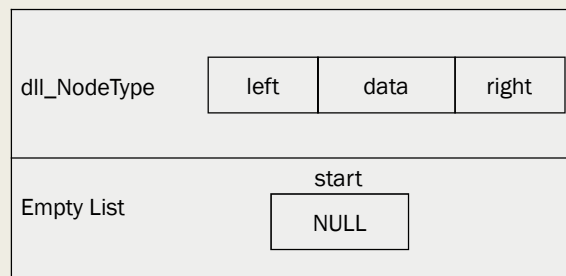
1. Initialize count with 0
2. Create a new node named temp of the structure NodeType
3. Assign the start to temp
4. Repeat until temp is NULL
 - (a) Increment count by 1
 - (b) Move to the next node by assigning next pointer of the temp to temp itself.
5. End.

Doubly Linked List (DLL)

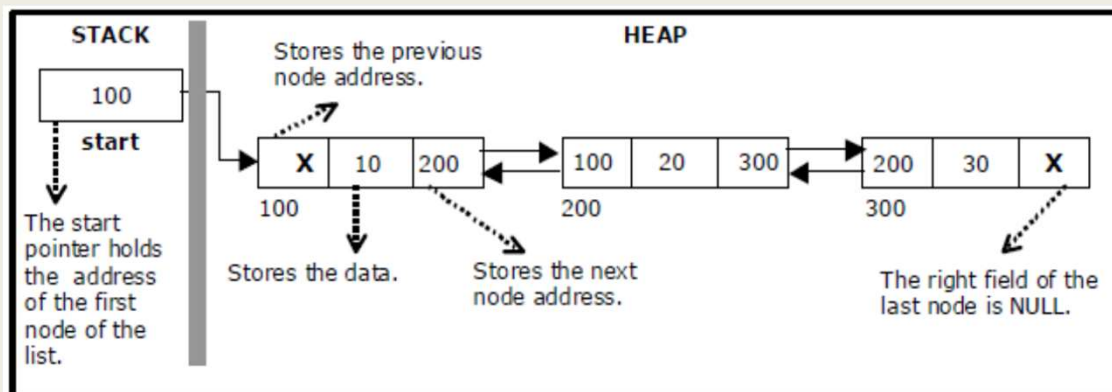
- A double linked list is a two-way list in which all nodes have two links. It provides bi-directional traversing. Each node contains three fields: (i) a "prev" or "left" pointer field which is a pointer used to link or connect to the previous node, (ii) a "data" field to store whatever element, and (iii) a "next" or "right" pointer field which is a pointer used to link or connect to the next node.
- Each node is allocated in the heap using malloc(), so the node memory continues to exist until it is explicitly de-allocated using free(). The Head of the list is a pointer to the "start / first" node of the list.

```

struct dll_Node
{
    struct dll_Node *left;
    int data;
    struct dll_Node *right;
};
typedef struct dll_Node dll_NodeType;
dll_NodeType *start = NULL;
  
```



Example of Doubly Linked List



Doubly Linked List – Basic Operations

- Creation
- Insertion
- Deletion
- Traversing in Both Directions
- Searching

Creating Node

Algorithm:

1. Allocate memory for the **structure of NodeType** and assign it to **newnode**.
2. Store the information in **data** field of the **newnode**.
3. Set the **left** and **right pointers** to **NULL**
4. Return the **address of the newnode**.
5. End

newnode		
X	1020	X
100		

Creation of Empty List

Algorithm:

1. Declare **start** pointer of the structure **NodeType**.
2. Set the **start** pointer with **NULL**.
3. End

Insertion of a Node

Steps:

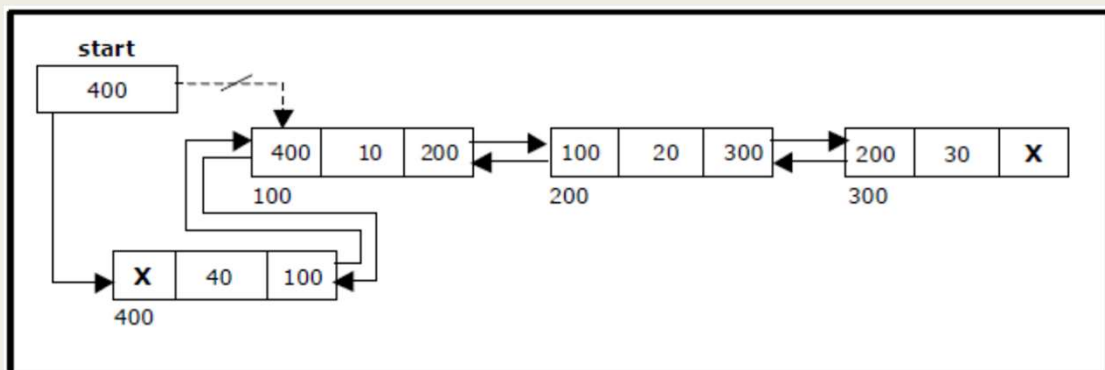
1. Create **newnode**.
2. If the list is empty (i.e. start is NULL) then assign the newnode to start.
3. else, if the newnode comes before the start node then
 - (a) *Set the right pointer of newnode with start*
 - (b) *Set the left pointer of start with newnode*
 - (c) *Assign the newnode to start*
4. else, traverse the list to reach at the node after which the newnode to be inserted and perform the following operations:
 - (a) *Set the left pointer of newnode with the curr_node itself*
 - (b) *Set the right pointer of newnode with the right pointer of the curr_node*
 - (c) *Set the left pointer of the right node of the curr_node with the newnode*
 - (d) *Assign the newnode to the right pointer of the curr_node*
5. End

Insertion of a Node

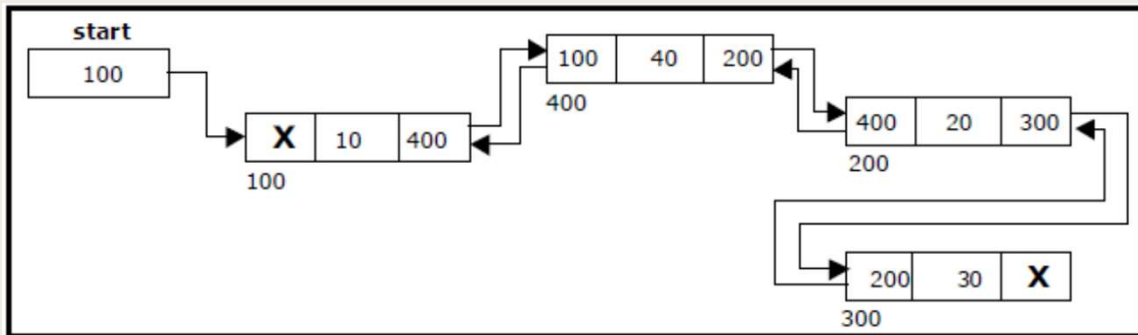
Algorithm:

1. newnode = CreateNode(data)
2. If start is NULL then start = newnode
3. else if the newnode comes before the start node then
 - (a) newnode->right = start
 - (b) start->left = newnode
 - (c) start = newnode
4. else, traverse the list to reach at the node (i.e. curr_node) after which the newnode to be inserted and perform the following operations:
 - (a) newnode->left = curr_node
 - (b) newnode->right = curr_node->right
 - (c) curr_node->right->left = newnode
 - (d) curr_node->right = newnode
5. End

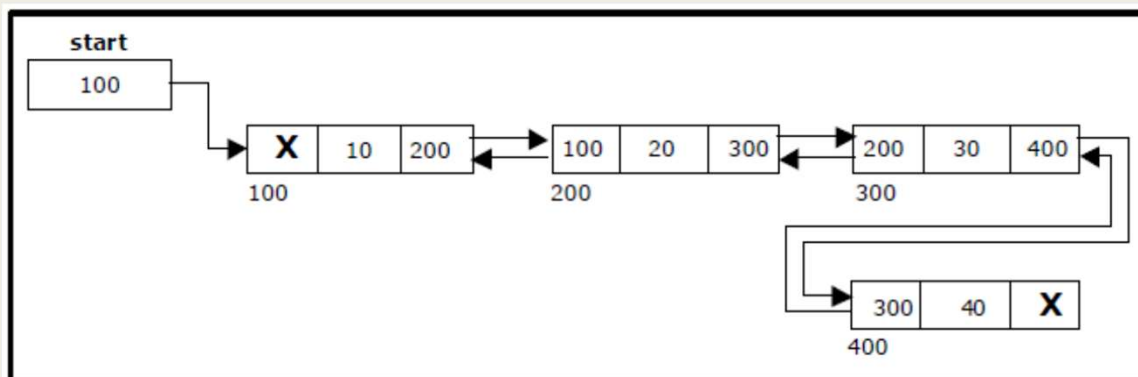
Insertion at the Beginning



Insertion at Intermediate Position



Insertion at the End



Deletion of a Node

Algorithm:

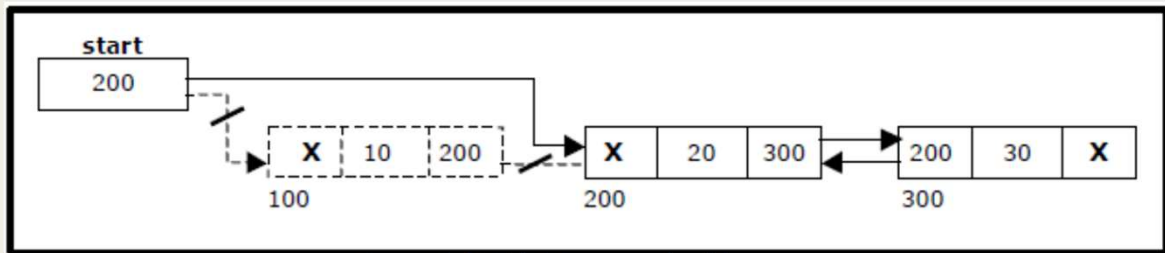
1. If start is NULL then Print "List is Empty" and Go to Step 4
2. Else If First node to be deleted then
 - (a) Assign the start to oldnode
 - (b) Assign right pointer of the oldnode to start
 - (c) Assign NULL to left pointer of start
 - (d) return oldnode
3. Else traverse to the list and reach to the Node (i.e. curr_node) just before the node to be deleted and perform:
 - (a) Assign the right pointer of the curr_node to oldnode
 - (b) Assign left pointer of the oldnode to the left pointer of the right node of the curr_node
 - (c) Assign right pointer of the oldnode to the right pointer of the curr_node
 - (d) return oldnode
4. End.

Deletion of a Node

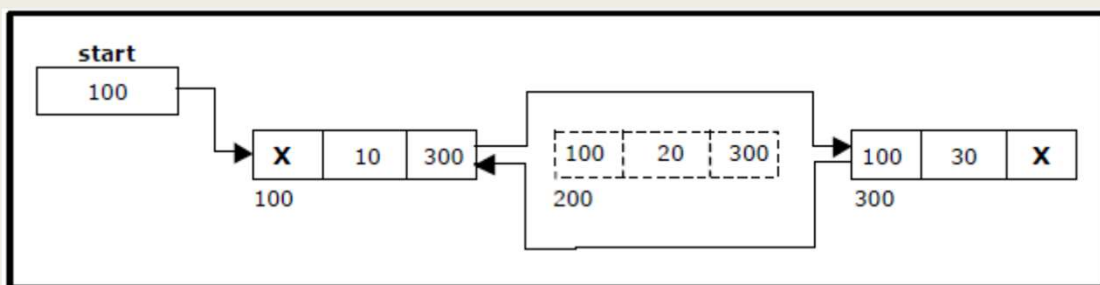
Algorithm:

1. If start is NULL then Print "List is Empty" and Go to Step 7
2. Else If First node to be deleted then
 - (a) oldnode = start
 - (b) start = oldnode->right
 - (c) start->left = NULL
3. Else traverse to the list and reach to the Node (i.e. curr_node) just before the node to be deleted and perform:
 - (a) oldnode = curr_node->right
 - (b) oldnode->right->left = oldnode->left
 - (c) curr_node->right = oldnode->right
4. data = oldnode->data
5. free (oldnode)
6. return data
7. End.

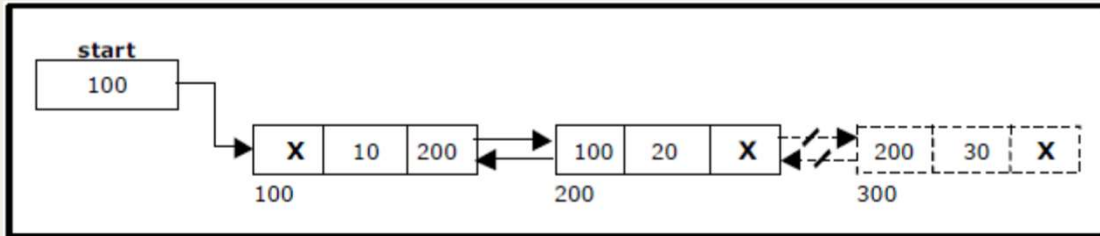
Deletion from the Beginning



Deletion from Intermediate Position



Deletion from the End



Traversing

Algorithm:

1. If start is NULL then Print "List is Empty" and Go to Step 4
2. Else
 - (a) Create a new node named temp of the structure NodeType
 - (b) Assign the start to temp
3. Repeat until temp is NULL
 - (a) Print data of the node temp
 - (b) Move to the right node by assigning right pointer of the temp to temp itself.
4. End.

Counting the Number of Nodes (i.e. Size of List)

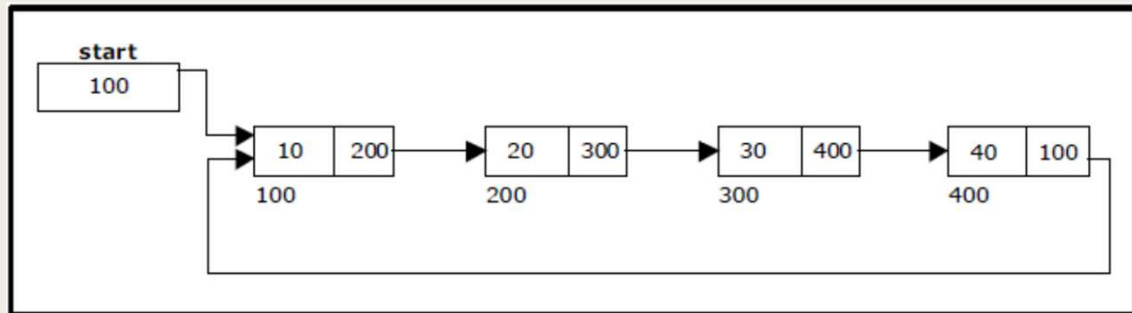
Algorithm:

1. Initialize count with 0
2. Create a new node named temp of the structure NodeType
3. Assign the start to temp
4. Repeat until temp is NULL
 - (a) *Increment count by 1*
 - (b) *Move to the right node by assigning right pointer of the temp to temp itself.*
5. End.

Circular Singly Linked List (C_SLL)

- It is an extension of singly linked list in which the next pointer field of the last node points back to the address of the first node.
- As it has no beginning and no end based on the links, it is necessary to have a special pointer called *start (or head)* pointer always pointing to the first node (or beginning) of the list.
- Circular linked lists are frequently used instead of ordinary linked list because many operations are much easier to implement.
- In circular singly linked lists, no null pointers are used, hence all pointers contain valid address.

Example of Circular Singly Linked List



Insertion of a Node

Algorithm:

1. `newnode = CreateNode(data)`
2. If `start` is NULL then `start = newnode` and set `newnode->next = start`
3. else if the `newnode` comes before the `start` node then
 - (a) `newnode->next = start`
 - (b) `start = newnode`
 - (c) Set the next pointer of the last node with `newnode`
4. else, traverse the list to reach at the node (i.e. `curr_node`) after which the `newnode` to be inserted and perform the following operations:
 - (a) `newnode->next = curr_node->next`
 - (b) `curr_node->next = newnode`
5. End

Deletion of a Node

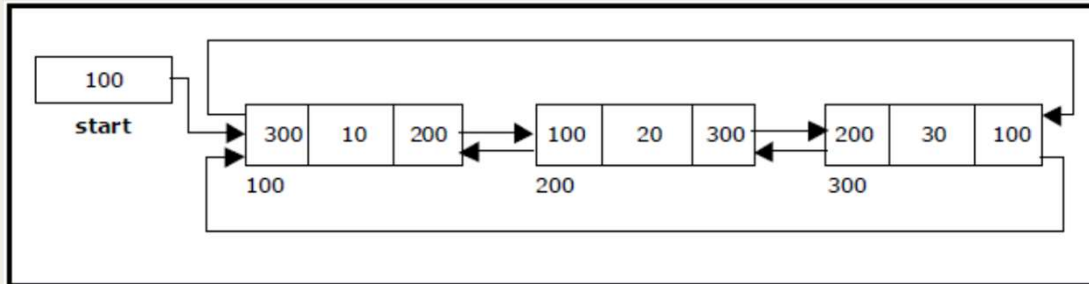
Algorithm:

1. If start is NULL then Print "List is Empty" and Go to Step 7
2. Else If First node to be deleted then
 - (a) `oldnode = start`
 - (b) `start = oldnode->next`
 - (c) Set the next pointer of the last node with start
3. Else traverse to the list and reach to the Node (i.e. `curr_node`) just before the node to be deleted and perform:
 - (a) `oldnode = curr_node->next`
 - (b) `curr_node->next = oldnode->next`
4. `data = oldnode->data`
5. `free (oldnode)`
6. return data
7. End.

Circular Doubly Linked List (C_DLL)

- It is also an extension of doubly linked list which has both successor pointer and predecessor pointer in circular manner.
- As it has no beginning and no end based on the links, it is necessary to have a special pointer called *start* (or *head*) pointer always pointing to the first node (or beginning) of the list.
- In circular doubly linked list the *right* link of the last node points back to the *start* node and *left* link of the first node points to the last node.
- The objective behind considering circular double linked list is to simplify the insertion and deletion operations performed on double linked list.
- In circular doubly linked lists, no null pointers are used, hence all pointers contain valid address.

Example of Circular Doubly Linked List



Insertion of a Node

Algorithm:

1. newnode = CreateNode(data)
2. If start is NULL then start = newnode and set newnode->right = start & newnode->left = start
3. else if the newnode comes before the start node then
 - (a) newnode->right = start
 - (b) start->left = newnode
 - (c) start = newnode
 - (d) Set the right pointer of the last node with newnode and left pointer of newnode with last node
4. else, traverse the list to reach at the node (i.e. curr_node) after which the newnode to be inserted and perform the following operations:
 - (a) newnode->left = curr_node
 - (b) newnode->right = curr_node->right
 - (c) curr_node->right->left = newnode
 - (d) curr_node->right = newnode
5. End

Deletion of a Node

Algorithm:

1. If start is NULL then Print "List is Empty" and Go to Step 7
2. Else If First node to be deleted then
 - (a) *oldnode = start*
 - (b) *start = oldnode->right*
 - (c) *start->left = NULL*
 - (d) *Set the right pointer of the last node with start and the left pointer of the start with last node*
3. Else traverse to the list and reach to the Node (i.e. *curr_node*) just before the node to be deleted and perform:
 - (a) *oldnode = curr_node->next*
 - (b) *oldnode->right->left = oldnode->left*
 - (c) *curr_node->right = oldnode->right*
4. *data = oldnode->data*
5. *free (oldnode)*
6. *return data*
7. End.

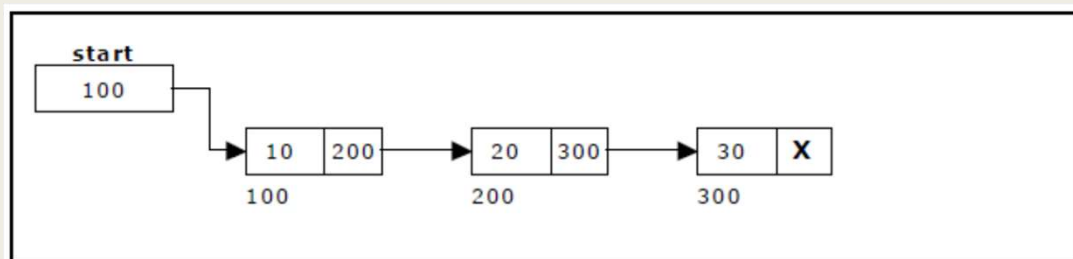
Comparison of Linked List

Sr. No.	Key	Singly linked list	Doubly linked list
1	Internal implementation / Node Structure	The node of singly linked list has two parts: (i) data and (ii) a pointer to the next node in the list	The node of doubly linked list has three parts: (i) a pointer to the previous node in the list, (ii) data and (iii) a pointer to the next node in the list
2	Memory consumption	It consumes lesser memory as it has only one pointer.	It uses more memory per node as it has two pointers.
3	Order of elements	Singly linked list allows traversal elements only in one way.	Doubly linked list allows element two way traversal.
4	Usage	Singly linked list are generally used for implementation of stacks	On other hand doubly linked list can be used to implement stacks as well as heaps and binary trees.
5	Index performance	Singly linked list is preferred when we need to save memory and searching is not required as pointer of single index is stored.	If we need better performance while searching and memory is not a limitation in this case doubly linked list is more preferred.
6	Complexity	Complexity of insertion and deletion at a known position is $O(n)$	Complexity of insertion and deletion at a known position is $O(1)$

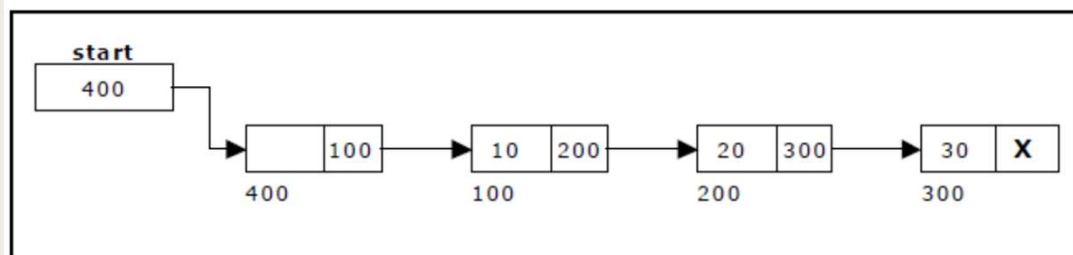
Header Linked List

- Also called as Linked List with a Header Node.
- A header node is a special node that is found at the beginning of the list.
- The header node need not represent the same type of data or structure as that of other nodes of the list.
- It is useful when information other than that found in each node is needed.
- For example, suppose there is an application in which the number of items in a list is often calculated. Usually, a list is always traversed to find the length of the list. However, if the current length is maintained in an additional header node that information can be easily obtained.
- Types:
 - **Grounded Header Linked List** in which last node contains null pointer
 - **Circular Header Linked List** in which last node points back to the header node

Header Linked List



Single Linked List without a header node



Applications of Header Linked List

- It simplifies the functions of adding and deleting elements in the list.
- In header node of linked list can maintain global properties of entire list and act as utility node:
 - In header node you can maintain count variable which gives number of nodes in list. You can update header node count member whenever you add /delete any node. It will help in getting list count without traversing in $O(1)$ time.
 - In addition to address of first node you can also store address of last node. So that if you want to insert node at rear end you don't have to iterate from beginning of list. Also in case of DLL(Doubly Linked List) if you want to traverse from rear end it helps.
- If you have a pointer in memory that points to the first element, then you need to pass the address of that pointer, rather than the value of that pointer, to the routines for list manipulation, because that pointer would need to change if an element were added or deleted at the head of the list. If that pointer, however, points to a special element in the list which is actually the pointer to the head of the list, then you do not need to pass the address of the pointer - you can pass its value, and it will never change. This technique "wastes" the data portion of that first element.

Array based Linked List

- Another alternative is to allocate the nodes in blocks.
- In fact, if the maximum size of a list is known, one can pre-allocate the nodes in a single array.
- The result is a hybrid structure – an array based linked list.
- Figure shows an example of null terminated single linked list where all the nodes are allocated contiguously in an array.

